Hdoc 46

1. Aim

To write a C program that reads an integer array, sorts its elements in ascending order using Bubble Sort, and finds the first and last occurrences of a target element using pointers and separate user-defined functions.

The program must display the target value and its first and last indices. If the target is absent, it must display `-1, -1`.

2. Algorithm

1. Start the program.
2. Read the size and elements of the integer array.
3. Pass the array's address to `Bubble_sort()` and sort it in ascending order.
4. Display the sorted array.
5. Call the `occurrence()` function and read the target element.
6. Search the sorted array to find the first and last occurrences of the target.
7. Display the target value and its corresponding indices.
8. If the target is not found, display `-1, -1`.
9. Stop the program.

3. Complete C Program

```
#include <stdio.h>

#define SIZE 100

void Bubble_sort(int *arr, int n);
void occurrence(int *arr, int n);

int main()
{
    int nums[SIZE], n, i;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    if (n <= 0 || n > SIZE)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    printf("Enter array elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    Bubble_sort(nums, n);

    printf("Sorted array: ");

    for (i = 0; i < n; i++)
    {
        printf("%d ", nums[i]);
    }

    printf("\n");

    occurrence(nums, n);

    return 0;
}
```

```
void Bubble_sort(int *arr, int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *arr, int n)
{
    int target, first = -1, last = -1;
    int i;

    printf("Enter target element: ");
    scanf("%d", &target);

    for (i = 0; i < n; i++)
    {
        if (*(arr + i) == target)
        {
            if (first == -1)
                first = i;

            last = i;
        }
    }

    if (first == -1)
    {
        printf("-1, -1\n");
    }
    else
    {
        printf("First occurrence: %d at index %d\n",
               target, first);
```

```
        printf("Last occurrence: %d at index %d\n",
               target, last);
    }
}
```

4. Step-by-Step Explanation

Step 1: Read the array

```
printf("Enter the size of array: ");
scanf("%d", &n);
```

The user enters the number of elements. The program then reads each integer into `nums[]`.

Step 2: Sort using Bubble Sort

```
if (*(arr + j) > *(arr + j + 1))
```

This compares two adjacent elements. If the left element is greater than the right element, they are swapped.

For example:

```
7 5 8 7 10
```

After sorting:

```
5 7 7 8 10
```

The expression `*(arr + j)` accesses an array element using pointer arithmetic. It is equivalent to `arr[j]`.

Step 3: Read the target element

```
printf("Enter target element: ");
scanf("%d", &target);
```

The user enters the value whose first and last occurrences must be found.

Step 4: Find the first occurrence

```
if (first == -1)
    first = i;
```

Initially, `first` is `-1`. When the target is found for the first time, its index is stored in `first`.

Step 5: Find the last occurrence

```
last = i;
```

Every time the target is found, `last` is updated. After the loop finishes, it contains the index of the final occurrence.

Step 6: Display the result

If the target is not present, both indices remain `-1`. Otherwise, the program displays the target and its first and last indices.

5. Sample Outputs

Example 1: Target occurs twice

Input:

```
Enter the size of array: 6
Enter array elements:
5 7 7 8 8 10
Enter target element: 8
```

Output:

```
Sorted array: 5 7 7 8 8 10
First occurrence: 8 at index 3
Last occurrence: 8 at index 4
```

Example 2: Target occurs once

Input:

```
Enter the size of array: 5
Enter array elements:
5 7 7 8 10
Enter target element: 8
```

Output:

```
Sorted array: 5 7 7 8 10
First occurrence: 8 at index 3
Last occurrence: 8 at index 3
```

Example 3: Target is absent

Input:

```
Enter the size of array: 6
Enter array elements:
5 7 7 8 8 10
Enter target element: 6
```

Output:

```
Sorted array: 5 7 7 8 8 10
-1, -1
```

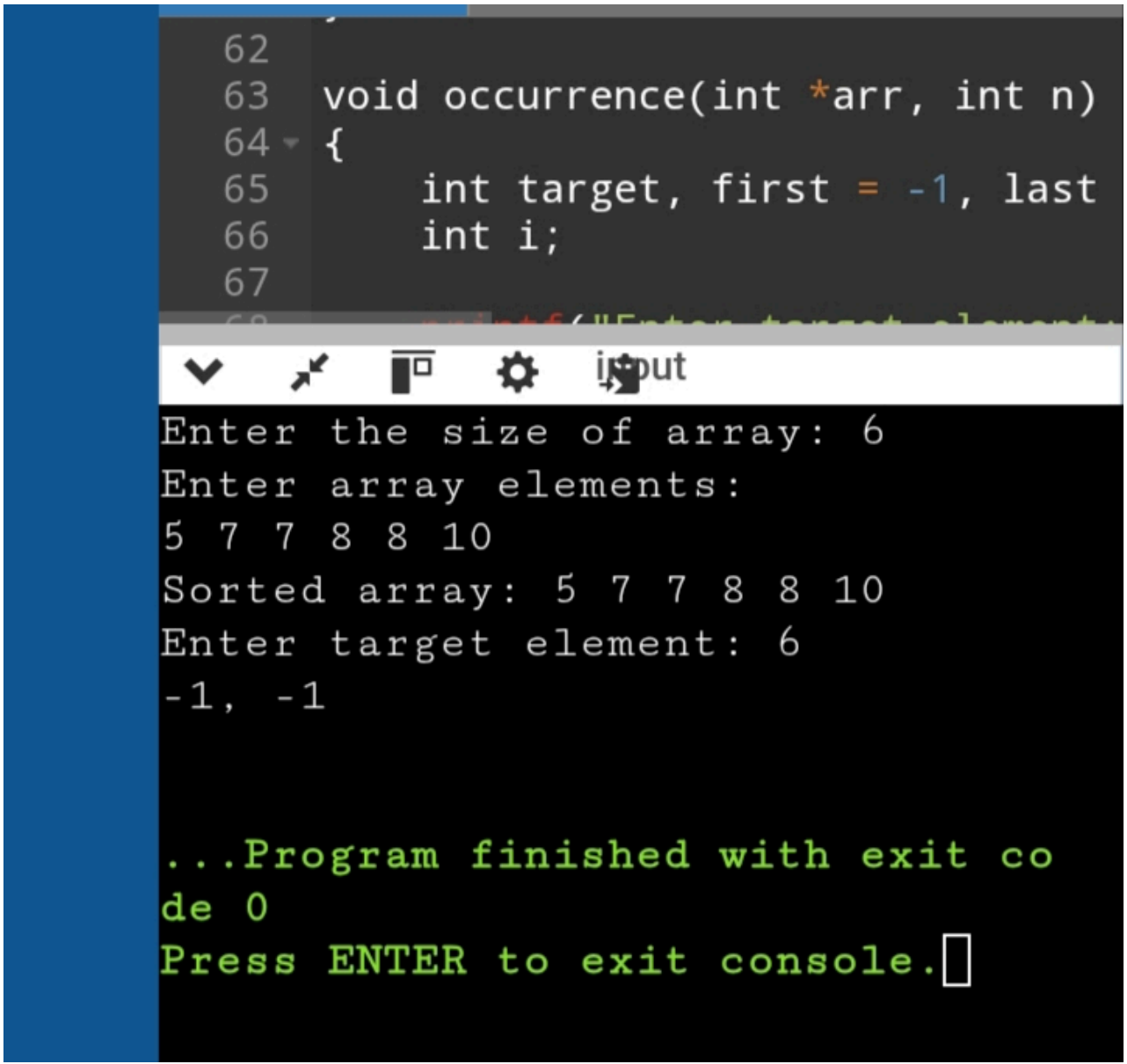
6. Important Functions Used

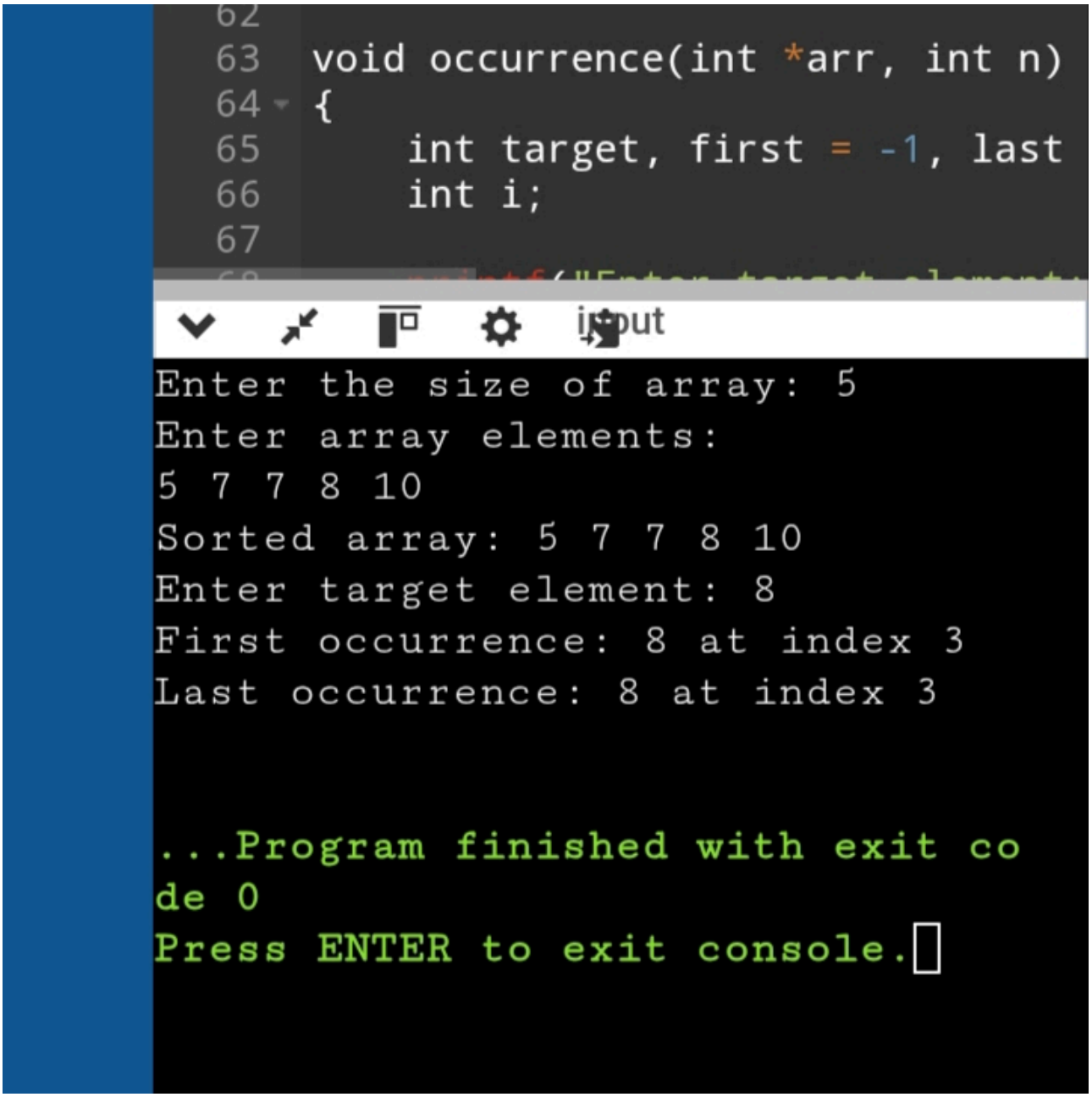
Function or concept	Purpose
`Bubble_sort()`	Sorts array elements in ascending order
`occurrence()`	Finds the first and last occurrences
`*(arr + i)`	Accesses an array element using a pointer
`scanf()`	Reads the array size, elements and target
`first` and `last`	Store the first and last indices

Note: Array indices start from `0`, not `1`. Therefore, in `5 7 7 8 8 10`, the two occurrences of `8` are at indices `3` and `4`.

7. Result

The C program successfully sorts an integer array in ascending order using Bubble Sort and finds the first and last occurrences of a target element using pointers and separate user-defined functions. If the target is absent, the program displays `-1, -1`.




62
63 void occurrence(int *arr, int n)
64 {
65 int target, first = -1, last
66 int i;
67
input
Enter the size of array: 5
Enter array elements:
5 7 7 8 10
Sorted array: 5 7 7 8 10
Enter target element: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 3
...Program finished with exit co
de 0
Press ENTER to exit console.

```
62
63  void occurrence(int *arr, int n)
64  {
65      int target, first = -1, last
66      int i;
67
```

input

```
Enter the size of array: 6
Enter array elements:
5 7 7 8 8 10
Sorted array: 5 7 7 8 8 10
Enter target element: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 4


...Program finished with exit co
de 0
Press ENTER to exit console.
```